Análisis Exhaustivo de la Arquitectura Hardware en el Backward Pass de Mamba: Optimización de Memoria Compartida y Fusión de Kernels CUDA

Introducción y Contexto Arquitectónico de los Modelos de Espacios de Estados

La evolución reciente en el campo del aprendizaje profundo y el modelado de secuencias de información densa ha estado dominada durante años por la arquitectura Transformer y su mecanismo central de atención autorregresiva. No obstante, la complejidad computacional cuadrática de la atención respecto a la longitud de la secuencia ha impulsado la búsqueda de arquitecturas subcuadráticas viables. En este panorama, los Modelos de Espacios de Estados (State Space Models o SSMs) estructurados han emergido como una alternativa matemáticamente rigurosa, capaz de procesar secuencias de longitud teóricamente infinita mediante inferencia en tiempo constante y entrenamiento de complejidad lineal.[1] La arquitectura Mamba, introducida por Albert Gu y Tri Dao, revoluciona este dominio al incorporar un mecanismo de selectividad dependiente de la entrada, permitiendo que el modelo filtre, memorice o ignore información dinámicamente a lo largo de la dimensión temporal.[1]

Esta selectividad de parámetros, sin embargo, destruye la propiedad de invariancia temporal (Linear Time-Invariant, LTI) que permitía a los modelos SSM anteriores utilizar transformadas rápidas de Fourier (FFT) y representaciones convolucionales extremadamente eficientes para el entrenamiento paralelo.[4] Al convertir los tensores paramétricos matriciales dependientes del tiempo, la evaluación del modelo requiere imperativamente un procesamiento recurrente. En un entorno de hardware moderno compuesto por Unidades de Procesamiento Gráfico (GPUs), la recurrencia secuencial estricta representa un antipatrón de diseño catastrófico que subutiliza drásticamente los miles de núcleos aritméticos disponibles y congestiona los buses de memoria.[5] Para resolver este aparente callejón sin salida, los autores de Mamba diseñaron una implementación a nivel de CUDA profundamente consciente de la arquitectura de hardware, inspirada conceptualmente en el éxito operativo de FlashAttention.[1]

El núcleo de esta innovación no reside únicamente en la propagación hacia adelante (forward pass), sino fundamentalmente en la ingeniería del algoritmo de propagación hacia atrás (backward pass) para el cálculo de gradientes durante el entrenamiento.[7] En la biblioteca mamba-ssm, específicamente dentro de los archivos de extensión de C++ y CUDA como selective_scan_bwd_kernel.cuh, se despliega una estrategia arquitectónica excepcional.[7] Este

informe proporciona un análisis exhaustivo y granular sobre cómo se orquestan los bloques de hilos de la GPU, cómo se gestiona y particiona de manera polimórfica la memoria compartida (__shared__) para resolver dependencias temporales inversas (desde el último paso temporal $t = L - 1$ hasta el inicial $t = 0$), y cómo se recalculan simultáneamente las variables continuas discretizadas junto con sus respectivos gradientes sin incurrir en cuellos de botella paralizantes en la memoria global.[2]

El Paradigma de la Rematerialización: Fusión de Kernels y Mitigación del Cuello de Botella HBM

El obstáculo más severo en el entrenamiento de modelos neuronales profundos contemporáneos no es la capacidad de procesamiento de operaciones de coma flotante por segundo (FLOPs), sino el ancho de banda de la memoria y el costo energético y de latencia asociado al movimiento de datos. La formulación de un modelo SSM selectivo recibe una entrada de dimensiones (B, L, D) —donde B representa el tamaño del lote de secuencias, L denota la longitud temporal de dichas secuencias, y D es la dimensión oculta del modelo— y proyecta esta entrada a un espacio de estados ocultos expandido de dimensionalidad espacial N.[5] Físicamente, esto significa que los cálculos intermedios generan tensores masivos con un requerimiento de forma (B, L, D, N).[9]

En el diseño convencional de grafos computacionales para diferenciación automática, marcos de trabajo como PyTorch almacenan en caché (guardan) las activaciones y estados intermedios completos del forward pass en la Memoria de Alto Ancho de Banda (HBM) del acelerador gráfico para poder aplicar la regla de la cadena posteriormente.[10] Si Mamba aplicara esta aproximación ingenua, materializar y almacenar el tensor de dimensionalidad temporal y espacial completa (B, L, D, N) incurriría en costos de Entrada/Salida (I/O) prohibitivos. La HBM, a pesar de su vasta capacidad volumétrica comparada con las memorias internas del chip, adolece de latencias de cientos de ciclos de reloj. Las operaciones de lectura y escritura necesarias para escribir el tensor de estados expandidos y luego recuperarlo en el backward pass saturarían los buses de memoria, asfixiando los Multiprocesadores de Transmisión (SMs) y provocando agostamiento aritmético, además de limitar fuertemente la longitud máxima de la secuencia antes de detonar errores de memoria insuficiente (Out-Of-Memory, OOM).[9]

La estrategia arquitectónica desplegada en el archivo selective_scan_bwd_kernel.cuh para neutralizar este colapso sistémico se basa en una técnica algorítmica clásica pero altamente optimizada: la rematerialización o recomputación acoplada con la fusión masiva de núcleos (Kernel Fusion).[6] El backward pass de Mamba se diseña para no leer los estados intermedios desde la HBM.[5] En su lugar, el kernel unificado lee de la memoria global únicamente los

tensores compactos que definen el contexto original: las entradas del modelo, los deltas de temporalidad (Δ), y los parámetros matrices fundamentales (A, B, C) y el gradiente de retropropagación proveniente de la capa superior (dout).[11]

Una vez que estos parámetros esenciales y compactos, cuyas dimensiones escalan meramente a proporciones manejables como (B, L, N) o (D, N), son cargados en la Memoria de Acceso Aleatorio Estático ultrarrápida (SRAM) del SM, los hilos computacionales proceden a recalcular (rematerializar) dinámicamente todos los estados intermedios del forward pass en el exacto momento en que se necesitan para calcular los gradientes inversos.[5] Este intercambio deliberado de ciclos computacionales adicionales a cambio de evadir transacciones de memoria globales asegura que el requerimiento espacial asintótico de memoria de Mamba permanezca idéntico al de un modelo Transformer estándar optimizado con FlashAttention, sosteniendo una complejidad de memoria que escala linealmente $\mathcal{O}(B \cdot L \cdot D)$ independiente de la expansión de estado algorítmica N.[6]

Reconstrucción Dinámica y Discretización ZOH en los Registros del Multiprocesador

Para que la recomputación en línea no imponga un retraso temporal que anule las ganancias obtenidas por evitar la HBM, la evaluación de la dinámica del sistema dentro del kernel debe ejecutarse con una sincronía matemática y de hardware prístina. Los parámetros del sistema de ecuaciones diferenciales continuas de un SSM deben ser transformados a su equivalente en tiempo discreto mediante una regla de discretización escalar dependiente de los pasos temporales Δ_t. Mamba adopta el mantenedor de orden cero (Zero-Order Hold o ZOH), cuya formulación analítica exacta establece las relaciones de transformación paramétrica para la matriz de evolución del estado $\bar{A}$ y el proyector de señales $\bar{B}$.[5]

Durante la iteración retrospectiva que analiza secuencias en bloques, la implementación en CUDA recalculada la discretización a nivel de los registros locales de cada hilo de procesamiento, evitando cualquier latencia de SRAM. Para la matriz de estado continua A, el cálculo de su forma discreta requiere la evaluación de la función exponencial sobre el producto del delta de paso y el propio parámetro: $\bar{A} = \exp(\Delta A)$.[8] Evaluar la función exponencial natural a gran escala, especialmente con tipos de datos de precisión compleja si los pesos lo dictan, resulta en micro-latencias operacionales por la naturaleza de las Unidades de Función Especial (SFUs) en la arquitectura de NVIDIA.

Para sortear esto con una elegancia técnica suprema, los ingenieros re-escalan el parámetro

continuo A mediante una multiplicación con una constante predefinida que cambia la base del exponente de e al sistema binario 2. El repositorio oficial instruye la inserción de una constante macro #define M_LOG2E 1.4426950408889634074 directamente en las cabeceras de compilación de la extensión CUDA.[8] Al multiplicar el valor original por M_LOG2E (logaritmo en base 2 de e), el kernel sustituye la invocación genérica de la exponencial por funciones de hardware intrínsecas hiper-optimizadas: exp2f(delta_vals[i] * A_scaled) para representaciones algebraicas reales, o cexp2f(delta_vals[i] * A_scaled) en el contexto de pesos de estado conjugados complejos.[8]

Simultáneamente a la evaluación del decaimiento temporal, el término responsable de incorporar la nueva entrada al estado oculto debe ser recalculado. Bajo la aproximación simplificada que Mamba emplea heurísticamente para la matriz interactiva, la discretización de la entrada evalúa $\bar{B}u \approx \Delta \cdot B \cdot u$.[8] El núcleo en C++ interroga estáticamente en tiempo de compilación a través de parámetros de plantilla si el tensor B es dinámico y dependiente de los datos (kIsVariableB). Si la matriz B varía en el tiempo debido al mecanismo de selectividad característico de Mamba, cada hilo indexado extrae su porción específica del array y recalcula el producto acumulado B_vals[i] * delta_vals[i] * u_vals[i] directamente en los registros.[8] Toda esta densidad aritmética de coma flotante ocurre sin la necesidad de un solo acceso a memorias superiores, preparando el terreno algorítmico para el escaneo temporal sin causar bloqueos (stalls) en el despacho del warp.

Topología y Partición Polimórfica de la Memoria Compartida (__shared__)

La memoria compartida en la plataforma CUDA (SRAM) actúa como un recurso crítico, finito e indispensable administrado a nivel del bloque de hilos. Ofrece latencias de alrededor de 20 a 30 ciclos de reloj, en agudo contraste con los más de 400 ciclos de la HBM, y un ancho de banda masivo cuando se accede sin provocar conflictos de bancos de memoria. En el selective_scan_bwd_kernel.cuh, la matriz de memoria __shared__ char smem_ se despliega dinámicamente y no obedece a un esquema de matriz lineal unidimensional simple. Por el contrario, se fragmenta espacial y temporalmente en una topología intrincada utilizando uniones superpuestas y alias estructurales del framework CUB (CUDA Unbound).[8]

La estrategia de estructuración se fundamenta en la reutilización de direcciones de memoria físicas para diferentes fases temporales del kernel, ahorrando espacio precioso de SRAM (kSmemSize) que de otro modo limitaría severamente la ocupación del Multiprocesador de Transmisión (SM Occupancy).[8] La memoria se subdivide lógicamente en los siguientes dominios fundamentales y especializados:

Alias de Dominio SRAM en CUDA	Capa de Abstracción Estructural (CUB / Custom)	Función en el Backward Pass y Mecánica de Reutilización
smem_load y smem_store	BlockLoadT::TempStorage, BlockStoreT::TempStorage	Facilita la carga y descarga coalescente de memoria global. Se superpone en ciclos de vida tempranos y tardíos del bloque iterativo donde las primitivas vectorizadas de acceso HBM son requeridas para leer y escribir gradientes consolidados dout, dA, etc.[8]
smem_reverse_scan	BlockReverseScanT::TempStorage	Específicamente reservada para resolver dependencias prefix-sum en reversa. Permite que el bloque interaccione temporalmente para generar los adjuntos del estado oculto a través de sumas de árbol binario de los hilos.[8]
smem_delta_a	Arreglo Transitorio Condicionado	Actúa como un caché de supervivencia entre los límites topológicos de los hilos. Transporta factores de atenuación exponencial del estado discretizado $\exp(\Delta A$ a través de divisiones de hilos.[8]
smem_running_postfix	Búfer Persistente de Fragmentos (Chunks)	Almacena el gradiente global que fluye lógicamente hacia atrás desde bloques temporales que ya han sido

		procesados, conectando un *chunk* analizado con su predecesor.[8]
smem_da y smem_dbc	Acumuladores Estacionales de Reducción	Destinados a recibir gradientes espaciales de los parámetros invariables en el tiempo A, B y C (BlockReduceT::TempStorage), sumando gradualmente los aportes derivados a lo largo del escaneo de secuencia antes del volcado a VRAM.[8]
smem_exchange	BlockExchangeT::TempStorage	Infraestructura vital para la manipulación geométrica de tensores. Muta los datos de un diseño de almacenamiento bloqueado a uno intercalado o *Striped* para habilitar transferencias coalescentes hacia la memoria HBM.[8]

Esta gestión particionada establece el andamiaje esencial sobre el que se ejecutarán simultáneamente decenas de miles de operaciones matemáticas complejas interdependientes temporalmente. La capacidad de superponer áreas como smem_load y los espacios para reducciones matemáticas es calculada en tiempo de compilación evaluando qué estructuras no se solapan en tiempo de vida (lifespan) gracias a un riguroso pre-cálculo manifestado en macros de tamaño condicional como kSmemIOSize, kSmemExchangeSize y kSmemReduceSize.[8]

Orquestación del Orden Temporal Inverso: Propagación de Estados a través de Chunks

El rasgo definitorio de un algoritmo SSM recurrente recae en que el estado en el instante t depende estrictamente de la acumulación del estado temporal en $t-1$. En el proceso de propagación de retroceso (backpropagation) del gradiente mediante la diferenciación implícita de la pérdida con respecto a los parámetros de la red, esta causalidad se invierte

dimensionalmente. Para calcular correctamente el gradiente de error sobre las activaciones tempranas en $t = 0$, el sistema requiere inexorablemente haber consolidado todos los gradientes propagados desde los pasos temporales futuros $t + 1$ hasta $t = L - 1$.[8]

Manejando secuencias extensas que típicamente sobrepasan con creces la capacidad de la memoria compartida local y la cardinalidad de subprocesos paralelos del SM, la implementación segmenta matemáticamente la longitud secuencial (L) en bloques lógicos contiguos referidos como "chunks".[14] Esta subdivisión, donde cada hilo asume el control de kNItems temporales consecutivos, se rige mediante un controlador algorítmico jerárquico expresado en el núcleo CUDA como un bucle iterativo de decrecimiento monotónico: for (int chunk = params.n_chunks - 1; chunk >= 0; --chunk).[8]

Esta dirección inversa decreta la ley fundamental de propagación de variables inter-bloque y de variables temporales. A medida que el bloque procesa retrospectivamente la información en reversa, emergen dos vectores de comunicación asíncrona altamente especializados:

En primera instancia, la **Comunicación Continua Discretizada inter-hilo**. Al interior de la unidad espacial de cada chunk, la variable estática almacenada y recalculada requiere integrarse globalmente a través de la topografía del bloque lógico de hilos.[8] La variable smem_delta_a es utilizada de manera magistral para sortear las colisiones o lecturas prematuras por discrepancia de sincronización de reloj en los hilos (conocido como Race Conditions). Para habilitar la fluidez sin imponer costosas barreras de sincronismo implacables tras cada ciclo computacional del reloj del GPU, la memoria recurre a una indexación condicionada par-impar lógica: state_idx + (chunk % 2) * MAX_DSTATE.[8] Esta técnica arquitectónica, conocida análogamente como técnica de *double-buffering* o mecanismo *ping-pong*, dicta que los hilos escriban las derivaciones finales atenuadas multiplicativas exponenciales del bloque temporal en una zona de la SRAM mientras asíncronamente leen las del bloque temporal precedente en la zona subyacente adyacente, proveyendo aislamiento semántico perfecto.[8]

En segunda instancia, la **Transferencia Residual de Estado y Gradientes de Cierre**. Finalizar el procesamiento algorítmico interno de un "chunk" genera derivadas adjuntas parciales acumuladas. Para continuar garantizando la derivación recursiva de la cadena de optimización, el fragmento de la historia latente del modelo (propagado a la inversa) debe entregarse íntegramente al "chunk" que lo precede cronológicamente en la secuencia.[19] Este conducto se formaliza en el espacio smem_running_postfix. Al arrancar el proceso de un nuevo segmento retrospectivo temporal, la inicialización del barrido extrae condicionalmente esta magnitud remanente, dictando formalmente variables de inicialización retrospectivas mediante operadores polimórficos funcionales. Este engranaje continuo y asíncrono permite computar una secuencia algorítmica matemática extendida sin interacciones con la API global que

generarían retrasos indeseados por el planificador de la GPU.[8]

Concurrencia Algorítmica: Escaneo Prefijo Hacia Adelante e Inverso Simultáneo

La resolución pura del bucle algorítmico inverso revela únicamente la gestión direccional del tiempo a nivel de control de flujo en la macroarquitectura. El desafío genuino reside en acelerar el núcleo secuencial interior en cada iteración del chunk, permitiendo a los hilos computar gradientes sin forzar a cada paso a esperar la respuesta estrictamente precedente. Aquí, la biblioteca Mamba expone la profundidad analítica del diseño subcuadrático explotando algoritmos geométricos paralelos.[4]

Al carecer de los estados intermedios del tensor de entrada (debido a la técnica de rematerialización), cada bloque de procesamiento CUDA asume el desafío dual de, simultáneamente y superponiéndose de forma espacial en el bloque de memoria: (1) Recrear la dirección del tiempo hacia adelante para inferir el estado transitorio del modelo, y (2) Procesar hacia atrás la señal de optimización del gradiente. Para lograr ambos propósitos concurrentemente a velocidades que desafían a FlashAttention, el kernel apela a primitivas de la biblioteca CUB (CUDA Unbound) para modelar "Parallel Prefix Sums" (Escaneos Prefijos Paralelos).[8]

La Primitiva de Recuperación del Estado Discreto (Forward Scan)

El estado oculto original para cada marcador temporal discreto dentro del límite del chunk actual (x_t) debe ser rederivado a partir de la matriz transitoria recalculada $\bar{A}$ y la contribución de entrada $\bar{B}u$.[8] Para acelerar dramáticamente el proceso en lugar de evaluar un bloque serial restrictivo en software for i = 1 to N, el código implementa un BlockScanT operando sobre el principio algorítmico paralelo Inclusivo (InclusiveScan).[8]

Alojado estructuralmente en la SRAM y orquestando comunicaciones cruzadas intra-warp a través del registro thread_data, la lógica CUB construye una red matemática virtual de reducciones de árbol binario, fusionando iterativamente secuencias de vectores en paralelo espacial de $\mathcal{O}(\log N)$ en latencia teórica frente al secuencial asintótico $\mathcal{O}(N)$ tradicional.[6]

Esta operación recobra de inmediato los vectores transitorios de estado x_t con una demanda temporal marginal que palidece frente a los cientos de microsegundos esperados en otras operaciones.[20]

La Primitiva Paramétrica del Gradiente Inverso (Reverse Adjoint Scan)

Inmediatamente, con los estados forward restaurados dinámicamente, la unidad funcional

pasa a evaluar los adyacentes del estado inverso. La variable dx_t representa el gradiente de la función de costo modelada en relación a los tensores transitorios internos de las redes Mamba.[8] Utilizando un operador diametralmente simétrico derivado de la configuración CUB, BlockReverseScanT emite una función intrínseca InclusiveReverseScan acoplada al área particionada smem_reverse_scan en la SRAM.[8]

Mediante esta superposición espacial y concurrencia secuencial imperativa en C++, el modelo no se ve constreñido por el dogma de un doble pase estricto general (recorrer todo de izquierda a derecha y luego todo de derecha a izquierda). Por el contrario, a escala micro-arquitectónica (por "chunk"), Mamba recobra la integridad forward y consolida los estados derivados backward virtualmente al mismo instante lógico empleando las matemáticas paralelas del hardware en un acoplamiento temporal perfecto que desarticula completamente los cuellos de botella clásicos.[5]

Cálculo Analítico y Acumulación de Gradientes Diferenciales

La obtención de las derivadas numéricas finales que requiere el ciclo de entrenamiento computacional del optimizador de la red profunda exige un análisis en múltiples ejes debido a la naturaleza heterogénea de las matrices implicadas. Dentro de los registros de los hilos, cada cálculo es descompuesto tras la obtención satisfactoria de las variables paramétricas locales (x_t y dx_t).[8]

1. **Gradiantes Estacionales y Constantes Matemáticas (dA)**: Para derivar la responsabilidad que tiene la matriz de progresión temporal constante A, la formulación del backward requiere ponderar la derivada transitoria con el impacto numérico residual aportado por la matriz previa. En el archivo fuente, Mamba computa un conjugado matemático denotado lógicamente como a_conj dentro de la sintaxis del lenguaje. El conjugado analítico emula restar la influencia de la proyección temporal de entrada $(\bar{B}u)$ respecto al estado recalculado temporal actual (x_t). La línea procedimental de la matemática en CUDA se enuncia como: dA_val += delta_vals[i] * dx * a_conj.[8] Al ser A un tensor constante independientemente de los tokens variables procesados en modelado LTI fundamental o su extrapolación inicial, sus derivadas son sumadas gradualmente en los registros temporales del ciclo unrolled del hilo (dA_val).
2. **Gradientes Condicionales Dependientes de la Data (dB y dC)**: Si la matriz B asume un marco dinámico parametrizado por la propia señal de secuencia (kIsVariableB dictaminado positivamente, dotando de selectividad real al estado), su tasa gradiente temporal no puede condensarse en una sumatoria global de secuencia. El gradiente es calculado instantáneamente y preservado secuencialmente en un array analítico:

dB_vals[i] = dx * delta_vals[i] * u_vals[i].[8]

3. **Tasa Gradiente Diferencial de la Ventana Temporal ($d\Delta$)**: El parámetro que rige la velocidad de muestreo o "step size" del entorno continuo Δ influye dualmente en la inercia del estado latente (A) y en la permeabilidad de la señal (B). Su acumulación diferencial refleja esta multiplicidad combinando sus trayectorias derivadas. La sintaxis del registro captura esta magnitud iterando de manera densa con la variable combinatoria: ddelta_vals[i] += ddelta_u * u_vals[i] + (dx * conj(A_val) * a_conj).real_.[8] Este sumario complejo enlaza cómo la variable de la discretización interviene sobre los espectros del costo escalar.[8]

Para garantizar que los parámetros matemáticos persistentes globalmente, como el acumulado dA_val de cada subproceso local, eviten contenciones operacionales o condiciones de escritura perjudiciales cuando converjan a un estado universal del modelo, la abstracción recurre a sumas estructuradas de paralelización reducida. Empleando la arquitectura definida por BlockReduceT (y extensiones polimórficas lógicas BlockReduceFloatT y BlockReduceComplexT para el marco algebraico de tensores reales e imaginarios, respectivamente), el sistema reduce de manera arbórea todos los cómputos fragmentados del bloque de subprocesos a una métrica absoluta singular unificada por bloque temporal.[8]

Estas reducciones de escala intermedia no colisionan contra la HBM. Como mecanismo intermedio, los resultados iterativos se condensan sistemáticamente al banco SRAM smem_da[state_idx], en donde subyacen latentes. La sintaxis condicional chunk == params.n_chunks - 1? dA_val : dA_val + smem_da[state_idx]; asegura perentoriamente que la transacción aritmética sumatoria proceda retrospectivamente y no efectúe solicitudes de latencia a la memoria de video del sistema hasta que se verifique flagrantemente el término totalitario de la cadena iterativa secuencial.[8]

Geometría de Tensores y Transformación de Layout: De Blocked a Striped

Uno de los logros de orquestación de hardware más sofisticados subyacentes en el kernel backward de Mamba es su mitigación proactiva contra la degradación de ancho de banda por operaciones desordenadas de memoria (uncoalesced memory access). El problema surge estructuralmente cuando los gradientes generados correspondientes a arreglos variables (dB y dC) deben finalmente escribirse a la HBM externa.

Debido a la topología interna del bucle del escaneo donde cada hilo en un warp ejecuta lógicamente sobre secuencias secuenciales de longitud kChunkSize, la disposición de la información que yace esparcida temporalmente a través de los registros del SM refleja un

diseño geométrico "Blocked" (Agrupado o Bloqueado).[21] En un arreglo Bloqueado espacial, el hilo lógico 0 posee las posiciones secuenciales 0 a $k-1$, el hilo lógico 1 posee las k a $2k-1$, etc.

Si estos hilos ordenaran comandos de volcado atómicos directamente a la VRAM, escribirían simultáneamente sobre direcciones de memoria altamente divergentes. Esto provocaría patrones transaccionales dispersos ("strided acceses"), obligando al bus de comunicación a generar múltiples ráfagas de transferencia ineficientes que ignoran líneas de caché y hunden brutalmente la eficiencia efectiva del ancho de banda de la GPU. Las operaciones de lectura de estas características exigen hasta n iteraciones de lecturas pesadas, penalizando severamente el modelado de secuencias masivas iterativas.[21]

Para subsanar esta fatalidad geométrica intrínseca a la paralelizacion recurrente selectiva, el bloque invoca transacciones mediadoras utilizando la segmentación estructurada de SRAM BlockExchangeT.[8] Cuando se detecta paramétricamente kIsVariableB o kIsVariableC en el compilador CUDA, se reservan cuotas de almacenamiento kSmemExchangeSize equivalentes a sizeof(typename BlockExchangeT::TempStorage) con previsiones específicas que duplican multiplicativamente su volumen para operaciones de espacios paramétricos algebraicos complejos (con factores binarios dictados como kNItems * 2 o kNItems respectivamente).[8]

Al culminar los diferenciales intermedios, todos los hilos coordinados invocan una matriz de transposición ejecutando la instrucción atómica virtual BlockExchangeT::BlockedToStriped.[8] En este marco simbiótico, cada subproceso inyecta iterativamente su array temporalmente aglomerado contiguo sobre la zona particionada smem_exchange y, acto seguido sincronizado asíncronamente a nivel bloque, procede a re-extraer los vectores computados intersecando ortogonalmente las direcciones a través del banco SRAM.

El resultado es una permutación morfológica que transforma al instante la estructura a un diseño "Striped" o Entrelazado (Warp-Striped Layout).[8] En esta nueva disposición estructurada ortogonalmente, hilos consecutivos de ejecución paralela dentro del mismo warp apuntan lógicamente a índices de memoria paramétricos físicamente adyacentes en vecindad. Posteriormente, las sucesivas interacciones masivas de escrituras directas mediante funciones superpuestas vectorizadas o rutinas gpuAtomicAdd de retorno logran volcar la carga matemática hacia el bus global en transacciones perfectamente coalescentes y contiguas.[8] Este nivel microscópico de ingeniería elude la penalización multiplicativa característica de las redes RNN entrenadas sincrónicamente en tarjetas tensoras contemporáneas, permitiendo escalabilidad a longitudes temporales estratosféricas sin degeneración asintótica.

Heurística de Lanzamiento del Kernel, Dimensiones de Grid y Ocupación del SM

La efectividad arquitectural expuesta previamente requiere ser instanciada con parámetros espaciales y configuraciones de clúster adaptativas en función de la longitud secuencial procesada. La ocupación teórica (Theoretical Occupancy) de los multiprocesadores se define limitantemente en gran medida por la demanda de la memoria dinámica compartida reservada per-thread, así como por las dimensionalidades topológicas que dictan la concurrencia espacial.[18]

En el módulo puente escrito en C++, esta asignación adaptativa geométrica se ejecuta mediante condicionales precompilados que modulan los templates macro selective_scan_bwd_launch. Este manejador jerárquico inyecta de manera automática los argumentos integrales $\langle$ kNThreads, kNItems $\rangle$ correspondientes a los vectores de hilos por bloque y la capacidad de elementos gestionados por unidad de subproceso respectivamente.[18]

Longitud Paramétrica Secuencial (params.seqlen)	**Hilos por Bloque Estructurado CUDA (kNThreads)**	**Carga Unitarias Iterativa por Hilo (kNItems)**
Secuencias Cortas $\leq$ 128	32	4
Rango Intermedio $128 < \cdots \leq$	32	8
Rango Ampliado $256 < \cdots \leq$	32	16
Secuencias Extendidas $512 < \cdots \leq$	64	16
Secuencias Masivas $>$	128	16

La configuración referenciada modela una heurística diseñada primordialmente para las microarquitecturas dominantes de NVIDIA que soportan las directivas intrínsecas del selective_scan_bwd_kernel, observándose paralelamente lógicas divergentes menos agresivas en bifurcaciones de código estructuradas para el entorno alternativo ROCm de arquitecturas de la corporación AMD (#ifndef USE_ROCM) donde tolerancias algorítmicas difieren

fundamentalmente por capacidades internas del compilador HIP subyacente.[18]

A un nivel macro-topográfico, los núcleos orquestan llamadas estructurales tridimensionales mediante coordenadas del clúster kernel<<<grid, Ktraits::kNThreads, kSmemSize, stream>>>(params) que asignan matrices de orquestación basadas explícitamente en el formato grid(params.batch, params.dim) (o factores divisionarios como params.dim / kNRows).[18] Esta malla algorítmica espacializa un bloque independiente para cada dimensión paramétrica de característica interior de red, a través del rango numérico del tamaño de batch, operando subsecuentemente procesos paralelos que logran escalar asintóticamente con los recursos de la máquina.

Para blindar la ejecución del núcleo de fallos silenciosos derivados de requerimientos de asignación transitorios insatisfechos en la GPU (Out of Memory transaccional), el tamaño pre-asignado a la memoria SRAM (kSmemSize) es evaluado empíricamente en tiempo de ensamblaje aglutinando la reserva matemática de componentes vectorizados abstractos: almacenamiento basal kSmemIOSize, transformaciones de arreglos dinámicos kSmemExchangeSize, requisitos matriciales analíticos algorítmicos dictados para resolver dependencias prefix-sum inclusivas directas e inversas por las bibliotecas locales, y unificaciones geométricas referenciadas lógicamente kSmemReduceSize.[8] Mediante el comando aplicativo intrínseco de hardware cudaFuncSetAttribute asignando la señal contextual cudaFuncAttributeMaxDynamicSharedMemorySize, el algoritmo garantiza concesiones directas con el gestor de software base mitigando disrupciones durante su propagación algorítmica.[18] En escenarios marginales que derivan fallos predecibles como el escalamiento disfuncional kDeltaSoftplus iterativo al no poseer el factor numérico subyacente limpio igual a ceros, los desarrollos subyacentes mitigan numéricamente este desborde compensando activamente la desviación lógica en el procesamiento vectorial con correcciones directas tras el filtro softplus.[14]

Conclusión sobre la Sinergia Hardware-Software en Mamba

El análisis sistemático del backward pass del modelo Mamba, estructurado y expuesto en las extensiones primigenias en CUDA del módulo selectivo y el análisis transaccional subsecuente, define indiscutiblemente uno de los avances más magistrales de codiseño contemporáneo entre la algoritmia del aprendizaje profundo y las ciencias de microarquitectura paralela. Superar el escollo inherente que sufren los Modelos de Espacios de Estados durante un requerimiento retroactivo (backpropagation) no habría sido mecánicamente posible únicamente desde una vertiente lógico-algebraica.

La estrategia arquitectónica exhibe cómo el desmantelamiento intencionado del paradigma clásico del framework PyTorch, prescindiendo del masivo almacenamiento global en el HBM a cambio de recomputación y discretizaciones efímeras in-situ (Rematerialization y M_LOG2E),

disuelve con eficacia drástica el limitante físico tradicional de latencia, ubicando a la arquitectura paralela a la memoria escalar que exhiben arquitecturas monolíticas como Transformers equipados con FlashAttention. Simultáneamente, el despliegue del almacenamiento compartido (SRAM) y su fragmentación polimórfica iterativa transaccional—sometida intermitentemente a esquemas de *double-buffering*, superposición de operaciones paralelas prefix directas/inversas (mediante algoritmos BlockScanT y BlockReverseScanT), y orquestaciones ortogonales manipulativas (con conversiones de métrica geométrica espaciales BlockedToStriped vía BlockExchangeT)—revelan una comprensión matemática integral de los regímenes de ocupación de hilos del hardware. El corolario fundamental es una formulación eficiente subcuadrática para modelos iterativos temporales, liberando a la recurrencia moderna de la degeneración del secuencialismo algorítmico, consolidando así infraestructuras altamente capacitadas de escalamiento ilimitado en secuencias del nivel de millones de tokens de información densa continua.

Obras citadas

1. state-spaces/mamba: Mamba SSM architecture - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba
2. [2312.00752] Mamba: Linear-Time Sequence Modeling with Selective State Spaces - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/abs/2312.00752
3. Mamba: Linear-Time Sequence Modeling with Selective State Spaces | OpenReview, fecha de acceso: marzo 4, 2026, https://openreview.net/forum?id=tEYskw1VY2
4. Here Comes Mamba: The Selective State Space Model | Towards Data Science, fecha de acceso: marzo 4, 2026, https://towardsdatascience.com/here-comes-mamba-the-selective-state-space-model-435e5d17a451/
5. Mamba: Make Sequence Models Fast Again | by Dong-Keon Kim - Medium, fecha de acceso: marzo 4, 2026, https://medium.com/@kdk199604/mamba-make-sequence-models-fast-again-540245a49155
6. Mamba: Linear-Time Sequence Modeling with Selective State Spaces - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2312.00752v2
7. selective_scan_cuda · Issue #605 · state-spaces/mamba - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/issues/605
8. mamba/csrc/selective_scan/selective_scan_bwd_kernel.cuh at main ..., fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/blob/main/csrc/selective_scan/selective_scan_bwd_kernel.cuh
9. Understanding Mamba and Selective State Space Models (SSMs) - Towards AI, fecha de acceso: marzo 4, 2026, https://towardsai.net/p/l/understanding-mamba-and-selective-state-space-models-ssms
10. Zero to Mamba: An intuitive explanation to the Mamba Architecture | by AI Club, IITM, fecha de acceso: marzo 4, 2026,

https://medium.com/@aiclub.iitm/zero-to-mamba-an-intuitive-explanation-to-the-mamba-architecture-d52265b771ab
11. Mamba on AMD GPUs with ROCm, fecha de acceso: marzo 4, 2026, https://rocm.blogs.amd.com/artificial-intelligence/mamba/README.html
12. Mamba: Linear-Time Sequence Modeling with Selective State Spaces - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/pdf/2312.00752
13. mamba/mamba_ssm/ops/selective_scan_interface.py at main - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/blob/main/mamba_ssm/ops/selective_scan_interface.py
14. Bug in selective scan backward · Issue #368 · state-spaces/mamba - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/issues/368
15. Win11系统中成功安装Mamba环境（causal-conv1d和mamba-ssm）的笔记原创 - CSDN博客, fecha de acceso: marzo 4, 2026, https://blog.csdn.net/tingyunye/article/details/147166838
16. Window 下Mamba 环境安装踩坑问题汇总及解决方法（无需绕过 selective_scan_cuda）原创, fecha de acceso: marzo 4, 2026, https://blog.csdn.net/yyywxk/article/details/140420538
17. ChatterjeeLab/PTM-Mamba at a476bbf - init commit - Hugging Face, fecha de acceso: marzo 4, 2026, https://huggingface.co/ChatterjeeLab/PTM-Mamba/commit/a476bbf22042f082e9a06fe36604f6bd1f2c9551
18. fecha de acceso: marzo 4, 2026, https://huggingface.co/kernels-community/mamba-ssm/raw/refs%2Fpr%2F3/selective-scan/selective_scan_bwd_kernel.cuh
19. Adding Initial Value Support to Selective Scan Forward Kernel · Issue #285 · state-spaces/mamba - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/issues/285
20. Optimizing the bwd pass of Mamba 2 · Issue #530 - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/issues/530
21. PackMamba: Efficient Processing of Variable-Length Sequences in Mamba Training - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2408.03865v2
22. arXiv:2408.03865v2 [cs.LG] 21 Aug 2024, fecha de acceso: marzo 4, 2026, https://arxiv.org/pdf/2408.03865?
23. Windows 下Mamba2 环境安装问题记录及解决方法（causal_conv1d=1.4.0 - CSDN博客, fecha de acceso: marzo 4, 2026, https://blog.csdn.net/yyywxk/article/details/143924743
24. Windows Support #12 - state-spaces/mamba - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/issues/12